Local AI Assistant

Technical Documentation Report

Phase 1: Inference Benchmarking | Phase 2: Structured Output Engineering | Phase 3: Model Comparison Study

Generated: 11 April 2026

Project Overview

This document records the engineering work completed across all three phases of the Local AI Assistant portfolio project. The goal is to build a production-pattern offline AI assistant using small language models running entirely on local hardware — with no cloud dependency, no API costs, and no data leaving the machine.

Problem Statement

Sending data to cloud-based LLM APIs is not always viable. Four constraints commonly rule it out in production:

- Privacy regulations — GDPR, HIPAA, and similar frameworks restrict sending sensitive data to third-party services.
- Latency requirements — network round trips add 200ms–2s of overhead, unacceptable in real-time applications.
- Cost at scale — API pricing compounds quickly; processing 1 million tokens per day can cost thousands of dollars monthly.
- Edge deployment — IoT devices, air-gapped systems, and remote deployments may have no reliable internet connectivity.

Technology Stack

Component	Technology	Purpose
Model Runtime	Ollama 0.20.5	Hosts and serves local LLM models

Models	llama3.2:3b · phi4-mini · mistral:7b	3B–7B parameter models for inference
Language	Python 3.11	Scripting and orchestration
Validation	Pydantic 2.12.5	Schema enforcement and validation
API Framework	FastAPI 0.135.3	REST API wrapper around Ollama
Terminal UI	Rich 14.3.3	Formatted tables and progress display
Operating System	Windows 11 — CPU only	No GPU acceleration

Phase 1 — Inference Benchmarking

Before building any application logic, we measure. This phase establishes a performance baseline for llama3.2:3b on the target hardware. These numbers inform every downstream engineering decision — model selection, timeout configuration, UX design, and cost estimation.

Metrics Captured

- Time to First Token (TTFT) — milliseconds from sending the prompt to receiving the first token. This is the latency a user perceives as a thinking pause.
- Total Response Latency — wall-clock seconds from request sent to full response received.
- Tokens Per Second — generation throughput, calculated as output_tokens / total_latency_s.

Raw Benchmark Results

#	Prompt	TTFT (ms)	Latency (s)	Tokens	Tok/s
1	What is the capital of France?	**8314.66**	8.728	8	0.92
2	Define recursion in one sentence.	3495.68	5.371	38	7.07
3	Explain list vs tuple in Python.	3596.72	22.154	357	16.11
4	What is gradient descent in ML?	3300.17	33.758	578	17.12
5	Write a Python prime check function.	3469.51	10.589	140	13.22
6	Explain TCP/IP in plain English.	3299.15	36.014	615	17.08
7	Main causes of the French Revolution?	3399.46	29.931	501	16.74
8	Write a Python string reverse function.	3393.49	5.823	49	8.41

9	Explain the CAP theorem.	3425.04	31.531	526	16.68
10	SQL vs NoSQL differences?	3441.63	30.023	506	16.85

Note: Prompt 1 shows elevated TTFT of 8314ms due to cold start — model loading into memory for the first time. All subsequent prompts settled at 3300–3600ms.

Summary Statistics

Metric	Average	P95
Time to First Token	**3,813 ms**	8,314 ms
Total Latency	**21.4 s**	36.0 s
Tokens Per Second	**13.0 tok/s**	—
Peak Throughput	**17.12 tok/s**	(prompt 4)
Hardware	CPU-only, Windows 11	No GPU

Engineering Observations

Cold Start vs Warm TTFT

Prompt 1 recorded 8,314ms TTFT — the model loading from disk into RAM. After this first call the model remains loaded and TTFT stabilises at 3,300–3,600ms. In production this is solved with a warmup call at server startup so no user ever feels the cold start penalty.

Throughput Warmup Effect

Short responses show 0.92–7.07 tok/s while long responses show 13–17 tok/s. This is not a hardware inconsistency — short responses complete before the model reaches full generation stride. The true hardware throughput baseline is 16–17 tok/s.

Latency Scales Linearly with Output Length

Total latency is directly proportional to output token count. Unbounded generation means unbounded latency. Any production system using this model must enforce a max_tokens limit to guarantee SLA compliance.

Phase 2 — Structured Output Engineering

Raw LLM output is an unstructured string. Production systems almost always require structured data — validated JSON objects with typed fields that downstream code can process reliably. This phase implements the constraint-validate-retry pattern used in real AI pipelines.

The Three-Layer Pattern

Layer	Mechanism	What it does
1	Constrained prompting	Instructs model to return only raw JSON with specified fields, types, and constraints. Includes a worked example.
2	Pydantic validation	Parses and validates the response against a strict schema. Rejects wrong types, missing fields, and invalid enum values.
3	Retry with context	On failure, re-prompts once with the exact validation error included. If second attempt fails, logs and moves on gracefully.

Pydantic Schema

```
class ConceptExplanation(BaseModel):
    topic: str
    difficulty: str     # validator: easy / intermediate / advanced
    explanation: str    # plain English, max 80 words
    has_code_example: bool
    one_line_summary: str  # validator: max 20 words
```

Temperature Experiment Results

Temperature	Successes	Failures	Notes
0.0 — deterministic	**1 / 5**	4 / 5	Markdown wrapping caused JSON parse failures
0.7 — stochastic	**5 / 5**	0 / 5	All succeeded, some required 2 attempts

Key Finding — Temperature 0.0 Reduces Instruction Following

At temperature 0.0, llama3.2:3b defaulted to wrapping its JSON in markdown code blocks in 4 out of 5 cases — its most rehearsed pattern for presenting structured data. At temperature 0.7, added randomness caused the model to skip the wrapper and return raw JSON directly. The retry mechanism recovered all remaining failures.

Engineering insight: For small models, temperature 0.0 can reduce instruction-following reliability. Always test structured generation across multiple temperature values before assuming 0.0 is the safest default.

Output Variance Across Temperatures

Temp	one_line_summary (recursion)	Observation
0.0	A function that calls itself to solve smaller versions of a problem.	Conservative, textbook phrasing
0.7	A self-calling function solves smaller versions of the same problem.	Rephrased — same meaning, different word order

---

Phase 3 — Model Comparison Study

Three models were evaluated on 40 standardised prompts across five task categories: factual recall, reasoning, code generation, creative writing, and instruction following. All models ran on the same CPU-only Windows 11 hardware with temperature=0.0 and seed=42 for reproducibility.

Models Under Test

Model	Parameters	Size on Disk	Character
llama3.2:3b	3B	2.0 GB	Fast, lightweight baseline
phi4-mini	3.8B	2.5 GB	Microsoft efficiency-focused model
mistral:7b	7B	4.4 GB	Larger model, higher quality target

Speed Comparison — All Models

Model	Avg Tok/s	Avg TTFT (ms)	Avg Latency (s)	Avg Word Count
llama3.2:3b	**14.55**	3,149	9.8	116
mistral:7b	7.42	3,452	23.6	119
phi4-mini	5.90	3,575	**85.2**	**772**

Quality Scores — All Models

Model	Overall	Factual	Reasoning	Code	Instruction
llama3.2:3b	**2.85**	3.00	2.50	2.75	3.00
mistral:7b	**2.92** ★	3.00	**2.88**	**3.00**	2.75

phi4-mini	**1.92**	3.00	**1.25**	**1.00**	1.62

Scale: 3 = Correct, complete, well structured | 2 = Correct but incomplete | 1 = Wrong or garbled

Key Findings

Finding 1 — mistral:7b is the best overall model

mistral:7b achieved the highest quality score of 2.92/3.0, with perfect scores on factual, code generation, and creative writing. Its larger 7B parameter count translates directly into better reasoning (2.88 vs 2.50 for llama) and more reliable code generation. The trade-off is speed — it runs at 7.42 tok/s vs llama's 14.55 tok/s — and memory, requiring 4.4 GB disk space vs 2.0 GB.

Finding 2 — phi4-mini's verbosity does not equal quality

phi4-mini generated an average of 772 words per response — 6.6x more than the other models — yet achieved the lowest quality score of 1.92/3.0. It scored only 1.25 on reasoning and 1.00 on code generation. More output does not equal more correct output. phi4-mini appears to have been trained for thoroughness over precision, which is a poor fit for structured evaluation tasks on CPU-constrained hardware.

Finding 3 — llama3.2:3b is the best speed-quality trade-off

llama3.2:3b achieved a quality score of 2.85 — only 0.07 below mistral:7b — while running at nearly 2x the speed (14.55 vs 7.42 tok/s). For latency-sensitive applications where response time matters more than marginal quality gains, llama3.2:3b is the clear choice. It also uses the least memory at 2.0 GB, making it the only model viable on severely resource-constrained edge hardware.

Finding 4 — All models perform equally on factual recall

All three models scored 3.0/3.0 on factual recall. Simple memorised facts do not require large model capacity. This confirms that model size only matters for generative, reasoning-heavy, or format-following tasks — not for factual retrieval where the answer is directly encoded in training data.

Tokens per Second by Category

Category	llama3.2:3b	phi4-mini	mistral:7b	Winner
Factual	13.86	11.21	7.39	**llama3.2:3b**
Reasoning	15.49	3.33	7.68	**llama3.2:3b**
Code	16.99	5.09	8.53	**llama3.2:3b**

Creative	13.47	4.41	6.94	**llama3.2:3b**
Instruction	12.94	5.48	6.56	**llama3.2:3b**

Model Selection Guide

Based on the benchmark results, here is a practical guide for choosing a model given specific deployment constraints:

Deployment Constraint	Recommended Model	Reason
Latency critical (<5s response)	**llama3.2:3b**	14.55 tok/s — fastest by 2x
Best quality, speed secondary	**mistral:7b**	2.92/3.0 quality score
Limited RAM (<4 GB available)	**llama3.2:3b**	Only 2.0 GB disk, lowest memory
Code generation tasks	**mistral:7b**	Perfect 3.0/3.0 on code category
Instruction following tasks	**llama3.2:3b**	3.0/3.0 on instruction category
Avoid phi4-mini when:	Precision required	1.92/3.0 overall — verbose but inaccurate

Key Engineering Learnings — All Phases

Finding	Production Implication
Cold start TTFT is 2.4x higher than warm	Always warmup model at server startup — never let user feel first-load latency
Throughput is 16–17 tok/s on CPU (llama)	Set max_tokens limits to cap worst-case latency for SLA compliance
Temperature 0.0 reduced JSON compliance	Test structured generation at multiple temperatures; 0.0 is not always safest for small models
Retry with error context achieved 100% recovery	Always include the exact validation error in retry prompts — vague retries rarely succeed
Pydantic caught all malformed responses	Never pass raw LLM output into application logic — always validate at the perimeter
mistral:7b best quality at 2.92/3.0	Larger models justify their resource cost for quality-critical tasks like reasoning and code

phi4-mini: 6.6x more words, lower quality	Verbosity is not a proxy for quality. Evaluate output correctness, not length
All models equal on factual recall	Model size only matters for generative and reasoning tasks — not memorised facts
llama3.2:3b best speed-quality trade-off	For edge/latency-constrained deployment, llama3.2:3b delivers near-mistral quality at 2x the speed

---

Quantization Analysis — Extra Mile

After completing the three-phase benchmark, a post-hoc investigation was conducted into the quantization format of all locally pulled Ollama models. This revealed a significant and non-obvious engineering finding.

Investigation Method

Ollama exposes a /api/show endpoint that returns model metadata including quantization level, format, parameter count, and internal digest ID. All five locally pulled models were queried and their digest IDs compared.

Model Inventory and Quantization Details

Model	Quantization	Format	Parameters	Size on Disk
llama3.2:3b	**Q4_K_M**	gguf	3.2B	2.0 GB
llama3.2:3b-instruct-q4_K_M	**Q4_K_M**	gguf	3.2B	2.0 GB
mistral:7b	**Q4_K_M**	gguf	7.2B	4.4 GB
mistral:7b-instruct-v0.3-q4_K_M	**Q4_K_M**	gguf	7.2B	4.4 GB
phi4-mini	**Q4_K_M**	gguf	3.8B	2.5 GB

Key Finding — Ollama Already Distributes Q4_K_M by Default

Every model pulled via Ollama is already distributed as GGUF Q4_K_M format. The digest IDs confirmed this conclusively: llama3.2:3b and llama3.2:3b-instruct-q4_K_M share an identical digest ID, as do mistral:7b and its explicit quantized variant. They are the same files on disk — pulling a model with an explicit quantization suffix downloads nothing new.

This means the entire Phase 1 and Phase 3 benchmark results already reflect Q4_K_M quantized performance. The 16 to 17 tok/s throughput measured on llama3.2:3b is quantized inference speed, which is the practically relevant metric for CPU-only deployment.

Full Precision vs Quantized — Memory Comparison

Model	Q4_K_M actual	F32 estimated	Memory saved
llama3.2:3b	2.0 GB	~12 GB	**6x less memory**
mistral:7b	4.4 GB	~28 GB	**6x less memory**

Estimated F32 sizes based on parameter count multiplied by 4 bytes per weight. Full precision mistral:7b would exceed available RAM on most consumer machines.

Quantization Format Comparison

Format	Bits/weight	Quality kept	vs F32 memory	Best use case
F32	32 bits	100% baseline	1x	Research only
F16	16 bits	~99.9%	0.5x	GPU with VRAM
Q8	8 bits	~99.5%	0.25x	High quality CPU
Q4_K_M	4 bits	**~96-98%**	**0.13x**	**CPU default**
Q2	2 bits	**~80-85%**	0.06x	Extreme constraints

Production Recommendations

- CPU-only edge deployment — always use Q4_K_M. Ollama provides this automatically. Near full-precision quality at a fraction of memory cost.
- GPU deployment with ample VRAM — consider F16 for marginally better quality on reasoning-heavy or code generation tasks.
- Extreme memory constraints under 2 GB RAM — Q2 is viable for factual recall only. Avoid for code generation or multi-step reasoning.
- Never use F32 on CPU — the memory and speed cost is prohibitive with negligible quality benefit over Q4_K_M.

Engineering insight: The most important quantization decision in production is not which level to choose — Ollama already makes the right default. The critical decision is which model size fits your hardware constraints, knowing all sizes will be Q4_K_M quantized automatically.

End of Report — Phases 1, 2 and 3 · Local AI Assistant Portfolio Project